UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

HENRIQUE MARQUES DIAS CHAVES

JHULIA ANDRADE DE SOUZA NOGUEIRA

FERRAMENTA EDUCACIONAL ORIENTATIVA PARA ALUNOS DE GRADUAÇÃO NA DISCIPLINA DE MANUFATURA INTEGRADA DA UTFPR

CURITIBA

2025

HENRIQUE MARQUES DIAS CHAVES

JHULIA ANDRADE DE SOUZA NOGUEIRA

FERRAMENTA EDUCACIONAL ORIENTATIVA PARA ALUNOS DE GRADUAÇÃO NA DISCIPLINA DE MANUFATURA INTEGRADA DA UTFPR

An Educational Guidance Tool for Undergraduate Students in the Integrated Manufacturing Course at UTFPR

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Engenharia Mecatrônica, da Universidade Tecnológica Federal do Paraná (UTFPR).
Orientador(a): Prof. Dr. Sidney Carlos Gasoto

CURITIBA

2025

4.0 Internacional

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

HENRIQUE MARQUES DIAS CHAVES
JHULIA ANDRADE DE SOUZA NOGUEIRA

FERRAMENTA EDUCACIONAL ORIENTATIVA PARA ALUNOS DE GRADUAÇÃO NA DISCIPLINA DE MANUFATURA INTEGRADA DA UTFPR

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Engenharia Mecatrônica, da Universidade Tecnológica Federal do Paraná (UTFPR).

Data de aprovação: 27/novembro/2025

_____
Sidney Carlos Gasoto
Doutorado
Universidade Tecnológica Federal do Paraná

_____
Luiz Carlos de Abreu Rodrigues
Doutorado
Universidade Tecnológica Federal do Paraná

_____
Márcio Alves Sodré de Souza
Doutorado
Universidade Tecnológica Federal do Paraná

CURITIBA

2025

RESUMO

A disciplina de Manufatura Integrada apresenta desafios significativos no processo de aprendizagem prática devido à quantidade limitada de equipamentos, ao tempo reduzido em laboratório e à exigência de conhecimento prévio em linguagem Ladder para a programação de Controladores Lógicos Programáveis (CLPs). Essas dificuldades prejudicam a execução do projeto final da disciplina, que envolve a programação de rotinas de um sistema de manufatura flexível (FMS). Este trabalho desenvolve uma ferramenta educacional orientativa para o planejamento e análise de projetos utilizando modelos SFC/GRAFCET, com o objetivo de apoiar o aprendizado anterior à programação em CLP. A ferramenta consiste em um algoritmo em Python para processamento e análise de arquivos PLCopenXML, integrado a uma interface web baseada em Flask. Também inclui um template que orienta os alunos na construção do GRAFCET no Codesys. O sistema realiza verificações automáticas de consistência estrutural, identificando erros e advertências para apoiar a correção e o aperfeiçoamento dos modelos. Os resultados demonstram que a ferramenta detecta adequadamente a maior parte das inconsistências e favorece uma aprendizagem mais autônoma e interativa no contexto da Manufatura Integrada.

Palavras-chave: GRAFCET; Python; Codesys; ferramenta educacional.

ABSTRACT

The Integrated Manufacturing course presents significant challenges in practical learning due to the limited availability of equipment, the reduced laboratory time, and the requirement for prior knowledge of Ladder language for programming Programmable Logic Controllers (PLCs). These factors hinder the execution of the course's final project, which involves programming routines of a Flexible Manufacturing System (FMS). This work develops an educational support tool for project planning and analysis using SFC/GRAFCET models, with the objective of assisting the learning process prior to PLC programming. The tool consists of a Python algorithm for processing and analyzing PLCopenXML files, integrated into a web interface built with the Flask framework. It also includes a template that guides students in the construction of GRAFCET in Codesys. The system performs automatic structural consistency checks, identifying errors and warnings to support the correction and improvement of project models. The results demonstrate that the proposed tool effectively detects most structural inconsistencies and promotes more autonomous and interactive learning within the context of Integrated Manufacturing.

Keywords: GRAFCET; Python; Codesys; educational tool.

LISTA DE ILUSTRAÇÕES

Figura 1 - A posição dos sistemas de manufatura em um grande sistema de produção……13
Figura 2 - Características de aplicação dos sistemas e células flexíveis de manufatura relativos a outros tipos de sistemas de manufatura……..15
Figura 3 - Sistema de controle de nível de tanque……16
Figura 4 - Programa de automação em linguagem Ladder……..18
Figura 5 - Contatos NA e NF em linguagem Ladder……..18
Figura 6 - Elementos básicos do GRAFCET……19
Figura 7 - Fluxograma do rascunho conceitual da ferramenta……...23
Figura 8 - Fluxograma para o desenvolvimento da ferramenta orientativa…….24
Figura 9 - Visualização do POU principal PLC_PRG do template no Codesys..27
Figura 10 - Processo de exportação do modelo como PLCopenXML no Codesys……...……..28
Figura 11 - Funções executadas pelo algoritmo……....…..30
Figura 12 - Árvore do projeto no Codesys……32
Figura 13 - SFC estruturado em etapas e transições……33
Figura 14 - Página web da ferramenta orientativa……..35
Figura 15 - SFC criado com base no template disponibilizado……..36
Figura 16 - Página de visualização dos resultados…….. 37
Figura 17 - Componente de visualização dos resultados - alerta……..38

LISTA DE QUADROS

Quadro 1 - Descrição das linguagens segundo a norma IEC 61131-3……..17
Quadro 2 - Método parse responsável por transformar o arquivo em uma estrutura de árvore python com nós individuais……29
Quadro 3 - Exemplo de namespace na tag principal <project>.....34

SUMÁRIO

1 INTRODUÇÃO 8
1.1 Problema 10
1.2 Objetivos 11
1.2.1 Objetivo geral 11
1.2.2 Objetivos específicos 11
1.3 Justificativa 12
1.4 Estrutura do trabalho 12
2 FUNDAMENTAÇÃO TEÓRICA 13
2.1 Conceitos básicos da manufatura integrada 13
2.2 Sistema Flexível de Manufatura (FMS) 14
2.3 Controlador Lógico Programável (CLP) 15
2.4 Linguagem Ladder 17
2.5 Sequential Function Chart (SFC) 19
2.6 Conversão GRAFCET/Ladder 20
2.7 Codesys e a linguagem Python 21
2.8 Interface Web 21
3 METODOLOGIA 23
3.1 Etapas do desenvolvimento da ferramenta 23
3.2 Revisão literária 25
3.3 Software para modelagem do SFC 25
3.4 Desenvolvimento do modelo GRAFCET 26
3.5 Escolha do método de transmissão/exportação 27
3.6 Desenvolvimento do algoritmo de processamento e análise 28
3.6.1 Leitura e estruturação do arquivo XML 29
3.6.2 Estrutura e funcionamento do algoritmo 29
3.7 Interação com o usuário 30
4 ANÁLISE DE RESULTADOS 32
4.1 Organização do template e estruturação do modelo GRAFCET 32
4.2 Processamento e interpretação do arquivo XML 33
4.3 Validação automática do modelo 34
4.4 Teste da ferramenta com modelos válidos e inválidos 35
5 CONCLUSÃO 39
REFERÊNCIAS 41
APÊNDICE A - Código do PLC_PRG do template criado no codesys 42
APÊNDICE B - Código do algoritmo analisador em python 45
ANEXO A - Trabalho de Avaliação dos componentes do FMS 52

1 INTRODUÇÃO

Entre as disciplinas ofertadas pelos cursos de engenharia da Universidade Tecnológica Federal do Paraná (UTFPR) do câmpus Curitiba, encontra-se manufatura integrada, ofertada no 8° período do curso de Engenharia Mecatrônica. É uma disciplina fundamental para a formação dos alunos, integrando conhecimentos de automação, programação de Controladores Lógicos Programáveis (CLPs) e sistemas de manufatura, preparando os alunos para desafios reais da indústria.

Um dos principais projetos da disciplina consiste em programar um Sistema de Manufatura Flexível (do inglês Flexible Manufacturing System, FMS) controlado por CLP, exigindo trabalho em equipe. No entanto, durante as práticas no laboratório, os alunos enfrentam dificuldades significativas ao interagir com, principalmente, a automação de armazém, também conhecida como sistema automatizado de armazenamento e recuperação (do inglês Automated Storage and Retrieval System, AS/RS) que compõe parte do FMS disponível.

Sugere-se que os fatores que contribuem para as dificuldades enfrentadas são, entre eles:

a) O uso de um software complexo para aprendizado, programação e operação;
b) A necessidade de manter todos os membros da equipe alinhados e atentos ao conteúdo e progresso das atividades;
c) Ciclos frequentes de tentativa e erro;
d) A disponibilidade limitada de máquinas, sendo que há apenas uma para operação.

Essas condições limitam o tempo disponível para os testes, gerando frustração e, em alguns casos, desistências.

Um ponto crucial é que, embora a metodologia recomendada pelo professor proponha iniciar o projeto do FMS com a modelagem em GRAFCET (acrónimo do francês Graphe Fonctionnel de Commande, Étapes Transitions), — uma ferramenta gráfica em que a ideia básica é a de separar um sistema ou processo em etapas e analisar uma etapa de cada vez —, muitos alunos ignoram essa etapa e partem diretamente para a programação em Ladder. Essa abordagem, embora hipoteticamente mais rápida, costuma resultar em erros de programação e retrabalho, aumentando a complexidade e o tempo necessário para a conclusão do projeto. A

falta de planejamento inicial via GRAFCET dificulta a identificação e correção de problemas, agravando os ciclos de tentativa e erro.

Além das dificuldades vistas, é importante considerar que no meio de aprendizado há grandes méritos na tentativa de torná-lo divertido e interessante com o auxílio de interação com o usuário para aumentar a eficácia do processo e gerar engajamento entre os estudantes.

Nesse contexto, ferramentas educacionais que permitem customização e adaptação têm se mostrado eficazes em promover um aprendizado mais dinâmico e significativo. Por exemplo, plataformas como Scratch e MATLAB/Simulink oferecem ambientes flexíveis onde os usuários podem criar e modificar projetos de acordo com suas necessidades. O Scratch, voltado para o ensino de programação, permite que alunos criem projetos interativos de forma visual, incentivando a experimentação e a criatividade. Já o MATLAB/Simulink é amplamente utilizado em engenharia para modelagem e simulação de sistemas, permitindo que os usuários adaptem modelos complexos de acordo com diferentes cenários. No entanto, essas ferramentas também apresentam limitações, como a necessidade de familiarização prévia com suas interfaces e a possível complexidade para iniciantes.

Com o intuito de aprimorar o processo de aprendizado dos estudantes e reduzir os atrasos causados pelos ciclos de tentativa e erro durante as atividades práticas, propõe-se a introdução de uma ferramenta educacional orientativa que auxilie o aluno no projeto de FMS utilizando o Gráficos de Funções Sequenciais (do inglês Sequential Function Chart, SFC), como ferramenta de planejamento. O SFC é derivado do GRAFCET, mas muitas vezes é referido como GRAFCET.

Uma abordagem promissora é a integração do Codesys (acrônimo do inglês Controller Development System), software de programação que permite criar programas de controle para sistemas de automação, com a linguagem Python, para que a partir de uma modelagem em SFC orientada, possa permitir a adaptação para a linguagem Ladder e a inclusão de mensagens/orientações informando o progresso do usuário.

É importante que essa ferramenta não engesse os alunos em um modelo fixo, mas sim permita a customização e a adaptação de acordo com as necessidades de cada projeto. A flexibilidade é essencial para que os alunos possam explorar sua criatividade, desenvolver soluções otimizadas e aprender com os erros de maneira construtiva. Ferramentas que permitem essa adaptação, como o LabVIEW e o

Tinkercad, têm sido amplamente utilizadas em contextos educacionais por oferecerem ambientes onde os usuários podem experimentar e modificar projetos livremente.

O LabVIEW, por exemplo, é uma plataforma gráfica de programação que permite a criação de sistemas de controle e automação de forma modular, enquanto o Tinkercad oferece um ambiente de simulação para projetos de eletrônica e robótica, permitindo que os alunos testem ideias antes de implementá-las fisicamente. Esses exemplos destacam a importância de uma ferramenta que seja ao mesmo tempo orientativa e flexível, promovendo um equilíbrio entre o aprendizado estruturado e a liberdade de criação.

1.1 Problema

Apesar da disciplina de manufatura integrada contar com um semestre letivo, quatro aulas semanais e horários extracurriculares para acesso ao laboratório de FMS — onde estão localizadas as máquinas utilizadas nas atividades —, múltiplos alunos não conseguem concluir com sucesso a atividade final, que representa a maior parte da nota para aprovação.

Para mitigar as dificuldades encontradas e liberar um tempo para auxiliar os alunos, os professores ministram o conteúdo em pares e se disponibilizam durante as práticas para esclarecer dúvidas e apontar erros. No entanto, as aulas teóricas consomem grande parte da carga horária, limitando o tempo disponível para os alunos colocarem em prática suas ideias durante o horário da aula.

Além disso, a disponibilidade de apenas uma máquina para operação obriga os alunos a revezarem seu uso, o que os leva a focar principalmente em revisar seus códigos em Ladder e à dedução das possíveis causas de problemas enquanto esperam suas rodadas para tentativa e erro.

Outro desafio é a necessidade de familiarização prévia com a linguagem Ladder e conhecimentos básicos sobre CLPs, conteúdos abordados em disciplinas anteriores, como redes industriais e sistemas supervisórios. O nível de preparo do aluno pode variar significativamente, influenciando em seu desempenho no decorrer do semestre. Alunos com menos experiência prévia tendem a enfrentar maiores dificuldades, enquanto aqueles com maior familiaridade podem se sentir desafiados a explorar soluções mais complexas e otimizadas.

Diante desses desafios, surge a proposta de desenvolver um modelo de ferramenta educacional que permita aos alunos trabalhar de maneira mais eficiente e diretamente ligada às disciplinas cujos conteúdos possam auxiliar no processo. A aplicação do SFC como ferramenta de planejamento pode facilitar o entendimento das etapas do processo e a transição para a programação em Ladder. No entanto, é crucial que essa ferramenta não se limite a um modelo fixo, mas permita a customização e a adaptação de acordo com as necessidades de cada projeto.

Portanto, a ferramenta educacional proposta deve ser flexível o suficiente para permitir que os alunos explorem diferentes abordagens, testem hipóteses e ajustem seus projetos conforme necessário. Isso não apenas facilita o aprendizado, mas também incentiva a inovação e a otimização, preparando os alunos para os desafios reais da indústria, onde a capacidade de adaptação e a criatividade são habilidades altamente valorizadas.

1.2 Objetivos

Os objetivos gerais e específicos relativos ao problema apresentado na seção anterior que orientarão o desenvolvimento do trabalho estão listados abaixo.

1.2.1 Objetivo geral

Desenvolver uma ferramenta educacional orientativa e flexível para auxiliar os alunos da disciplina de Manufatura Integrada no projeto de FMS, utilizando o SFC/GRAFCET como ferramenta de planejamento.

1.2.2 Objetivos específicos

Para obter os resultados propostos pelo objetivo geral, os objetivos específicos foram definidos:

a) criar um template que permita aos alunos adicionar, remover ou modificar etapas e transições de acordo com as necessidades de cada projeto;
b) desenvolver um algoritmo que identifique inconsistências ou erros de estrutura nos modelos criados pelos alunos, fornecendo orientações claras e sugestões de melhorias;

c) projetar uma interface web que disponibiliza orientações, um template base e análise automatizada do GRAFCET, que promovem a aplicação prática e a adoção da ferramenta em ambiente educacional.

1.3 Justificativa

No cenário de educação, o processo de aprendizado pode ser dificultado pelos aspectos abstratos encontrados nos conteúdos lidados em sala, o que pode ser contornado com a assimilação de novos objetos ou ideias para que os estudantes possam se interessar e interagir com os meios (Piaget, 1970).

Seguindo o princípio acima, a introdução de uma plataforma onde os alunos possam interagir com os modelos através de representações simples e ter acesso a um assistente digital é um meio efetivo de ensino. Ao trazer uma ferramenta introdutória relacionada ao conteúdo a ser tratado e incentivar a interação entre os alunos e orientadores através de resolução de dúvidas, o trabalho em equipe e a assimilação do problema são desenvolvidos de maneira eficaz para o alcance de sua resolução (Moraes, 2010).

Esta pesquisa tem como propósito explorar métodos inovadores para o desenvolvimento de um modelo educacional que facilite o aprendizado em manufatura integrada. A ferramenta promove a colaboração entre professores e alunos, criando um ambiente de aprendizado mais interativo e eficiente. Além disso, ao reduzir os ciclos de tentativa e erro e fornecer orientações, a proposta contribui para um processo de ensino mais dinâmico e alinhado às necessidades da disciplina.

1.4 Estrutura do trabalho

O Capítulo 1 apresenta o problema, a justificativa e os objetivos a serem alcançados neste trabalho. O Capítulo 2 possui o referencial teórico usado para referenciar os temas do trabalho, passando por temas como FMS, SFC e Python.

O Capítulo 3 discorre sobre a metodologia utilizada.

O Capítulo 4 apresenta as análises e discussões acerca dos resultados obtidos. Por fim, no Capítulo 5 encontram-se as considerações finais e melhorias futuras.

2 FUNDAMENTAÇÃO TEÓRICA

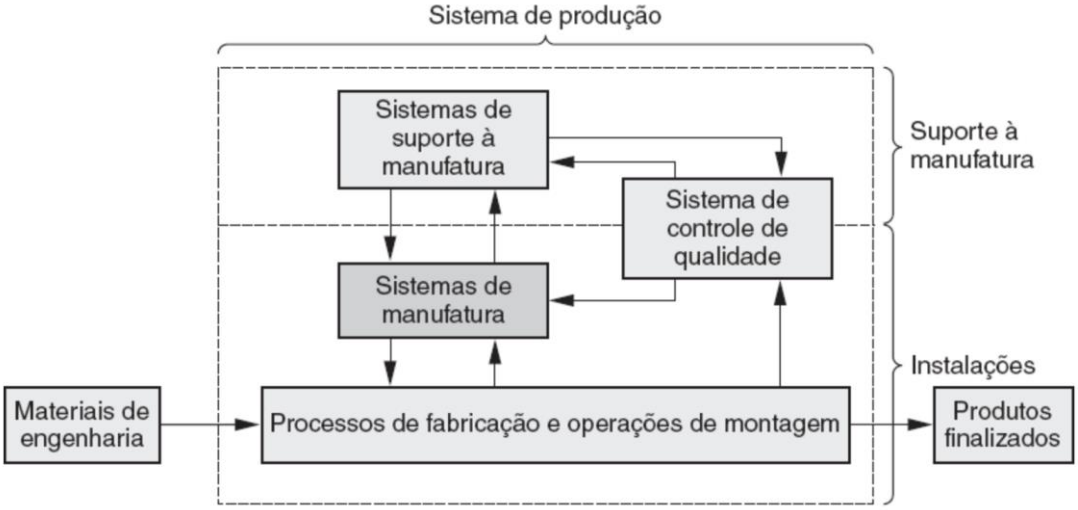
Este capítulo apresenta a pesquisa bibliográfica dos assuntos que envolvem o desenvolvimento de uma ferramenta educacional orientativa que auxilie o aluno no projeto de FMS.

2.1 Conceitos básicos da manufatura integrada

Um sistema de manufatura pode ser definido como um conjunto de equipamentos e recursos humanos integrados que realiza uma ou mais operações de processamento e/ou montagem na matéria-prima, na peça ou em conjunto de peças. O equipamento integrado consiste em máquinas de produção, dispositivos de manuseio e posicionamento de materiais e sistemas computadorizados (Groover, 2017).

Como ilustrado na Figura 1, os sistemas de manufatura são encontrados no chão de fábrica, desempenhando um papel central no processo de produção. A integração entre máquinas, sistemas de controle e recursos humanos é essencial para a eficiência das operações industriais.

Figura 1 - A posição dos sistemas de manufatura em um grande sistema de produção



Fonte: Groover (2017)

Os sistemas integrados de manufatura englobam algumas configurações, destacam-se as linhas de produção manuais ou automatizadas, células de manufatura e o FMS. Caracterizam-se pela presença de múltiplas estações de trabalho e/ou máquinas, interconectadas por um subsistema de manuseio de materiais com o

objetivo de transportar as peças/produtos entre as estações, trazendo fluidez para o processo produtivo. Grande parte desses sistemas usam controle computadorizado que coordena as ações das estações e máquinas que permitem o monitoramento do desempenho global do sistema em busca de eficiência operacional.

2.2 Sistema Flexível de Manufatura (FMS)

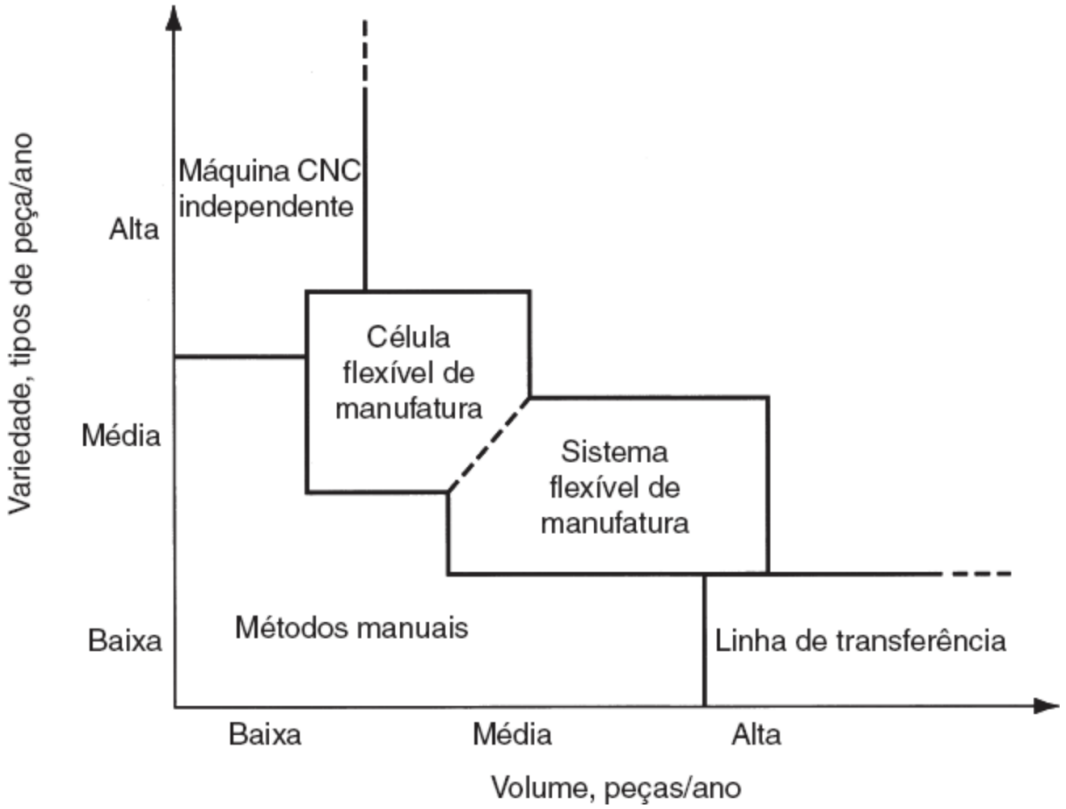
O FMS inclui um computador central com interfaces com outros componentes de hardware. Além do computador central, cada máquina e demais componentes geralmente têm microcomputadores para controle de suas unidades individuais. A função do computador central é coordenar as atividades dos componentes para alcançar boa operação global do sistema. Ele realiza essa função por meio de software. (Groover, 2017)

O software de um FMS é composto por módulos associados às diversas funções realizadas pelo sistema. Alguns exemplos são:

a) download de programas de CNC (Controle Numérico Computadorizado): envio de instruções para máquinas-ferramentas;
b) controle do sistema de manuseio de materiais: gerenciamento do transporte de peças entre estações;
c) gestão de ferramentas: organização e substituição de ferramentas nas máquinas.

Os FMSs são utilizados tipicamente para a produção em médio volume e variedade intermediária, como visto na Figura 2.

Figura 2 - Características de aplicação dos sistemas e células flexíveis de manufatura relativos a outros tipos de sistemas de manufatura



Fonte: Groover (2017)

2.3 Controlador Lógico Programável (CLP)

O CLP é adequado ao ambiente industrial, por ser resistente a ruídos, poeira, umidade e perturbações eletromagnéticas, além de apresentar pequenas dimensões. Em outras palavras, o CLP pode ser visto como um computador, dotado de interface amigável com o usuário, cuja função é executar controle de processos com vários níveis de complexidade (Roque, 2014).

Os elementos mais importantes de um CLP são as entradas, as saídas e a memória interna. Somente por meio de suas entradas o CLP recebe informações do mundo externo. De forma similar, ele só pode controlar algum dispositivo se este estiver conectado em uma de suas saídas (*Franchi et al.* , 2020).

Algumas funções específicas do CLP são sequenciamento, registro, lógica, temporização, operações aritméticas e contadores. Possui módulos de entrada/saída digitais ou analógicas.

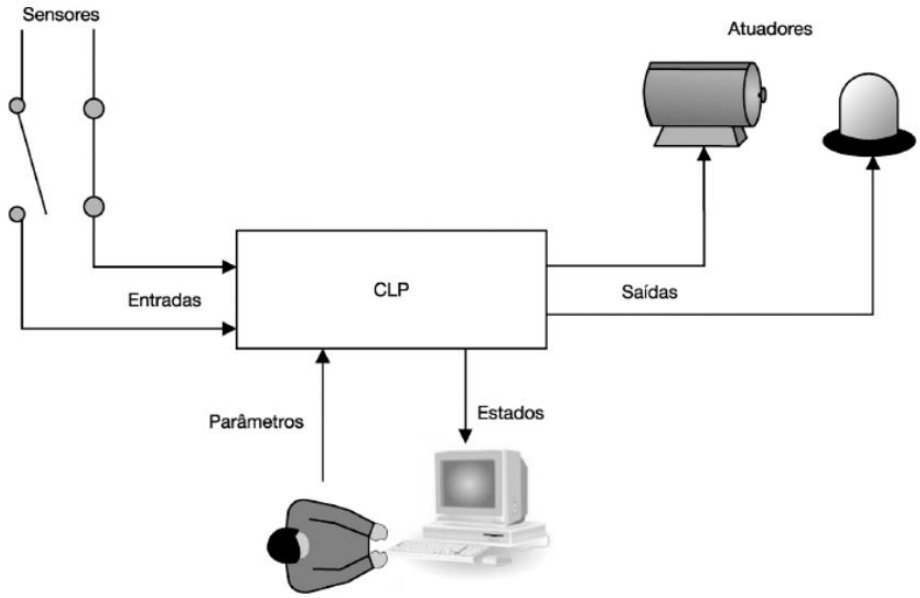
Os sistemas de controle são divididos em três partes:

a) Sensores: dispositivos que detectam estímulos físicos e transmitem sinais elétricos correspondentes através de redes industriais de comunicação. Esses sinais são enviados à entrada do CLP, que os processa conforme o programa armazenado em sua unidade central de processamento.
b) Atuadores: convertem os sinais elétricos do CLP em ações físicas, ligando ou desligando componentes do processo, como motores, válvulas, sinalizadores ou esteiras. Eles são conectados às saídas do CLP.
c) Controladores: o CLP utiliza um programa para calcular operações com base nos sinais de entrada e atuar sobre as saídas. Ele monitora processos em tempo real, recebendo sinais dos sensores e enviando comandos aos atuadores para controlar o processo de acordo com os parâmetros definidos.

Toda planta ou todo processo industrial necessita de algum tipo de controlador, para garantir uma operação segura e economicamente viável (Roque, 2014).

A figura 3 exemplifica o funcionamento de um sistema de controle.

Figura 3 - Sistema de controle de nível de tanque



Fonte: Roque (2014)

2.4 Linguagem Ladder

A Norma IEC 61131-3 — norma internacional que estabelece padrões para a programação de CLPs —, definiu cinco linguagens de programação para controladores lógicos programáveis (Franchi et al., 2020).

a) diagrama de blocos de funções (do inglês *Function Block Diagram, FBD*);
b) linguagem Ladder (do inglês *Ladder Diagram, LD*);
c) sequenciamento gráfico de funções (do inglês *Sequential Function Chart, SFC*);
d) lista de instruções (do inglês *Instruction List, IL*);
e) texto estruturado (do inglês *Structured Text, ST*).

Três destas são gráficas e duas são textuais, conforme demonstrado no Quadro 1.

Quadro 1 - Descrição das linguagens segundo a norma IEC 61131-3

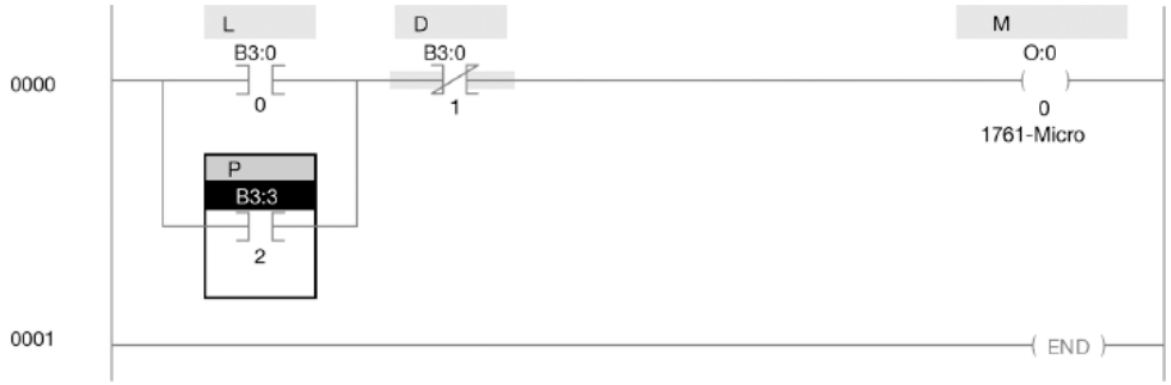
Texto Estruturado (ST)	**Textuais**
Lista de Instruções (IL)	
Diagrama de Blocos de Funções (FDB)	**Gráficas**
Linguagem Ladder	
Sequenciamento Gráfico de Funções (SFC)	

Fonte: Adaptado de Franchi *et al.* (2020)

A linguagem Ladder é uma linguagem gráfica muito utilizada, baseada na lógica de contatos virtuais, cujo programa aciona bobinas de saídas segundo valores aferidos por sensores ou chaves conectados à entrada dos CLPs, com o intuito de ligar motores, lâmpadas ou realizar contagem, temporização ou controles proporcionais, integrais e derivativos. (Roque, 2014)

Na figura 4 encontra-se um exemplo de código em Ladder em que L, D e P são contatos virtuais, enquanto M é a saída.

Figura 4 - Programa de automação em linguagem Ladder



Fonte: Roque (2014)

A Figura 5 traz a simbologia de contatos normalmente abertos (NA) e normalmente fechados (NF), utilizada na linguagem Ladder.

Figura 5 - Contatos NA e NF em linguagem Ladder

Fonte: Roque (2014)

Um programa em Ladder descreve funções lógicas através de contatos e relés, representando os pólos positivo e negativo de uma bateria por meio de duas barras verticais. O aspecto visual do ambiente de desenvolvimento integrado é semelhante a uma escada e, por esse motivo, batizou-se essa linguagem de Ladder, que significa degrau em inglês (Roque, 2014).

Cada degrau é composto por combinação entre condições de entrada (representadas por contatos NA e NF) e uma instrução de saída no final da linha.

Alguns softwares usados para programar em Ladder são: WEG Ladder Programmer (WLP), Winproladder, OpenPLC, EasyPLC Software Suite, entre outros. O software utilizado no laboratório da UTFPR é o CX Programmer, um software proprietário da Omron que permite programar o modelo de CLP CJ1M que está instalado no FMS.

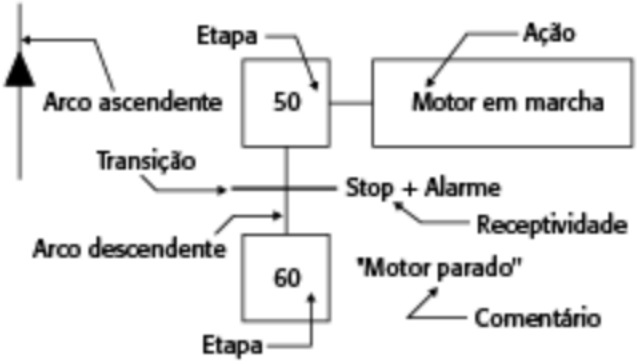
2.5 Sequential Function Chart (SFC)

O SFC mais conhecido como GRAFCET, surgiu em 1977 em um grupo de trabalho da *Association Française pour la Cybernétique Économique et Technique* (AFCET – Associação Francesa para a Cibernética Econômica e Técnica). Em junho de 1982 foi criada a norma francesa UTE NF C 03-190 (diagrama funcional Grafcet para a descrição dos sistemas lógicos de comando). Posteriormente, foi criada uma linguagem com base no GRAFCET, chamada de SFC (do inglês *Sequential Function Chart* – sequenciamento gráfico de funções)(Franchi *et al*., 2020).

É uma linguagem que permite a descrição de ações sequenciais, paralelas e alternadas existentes numa aplicação de controle. As unidades organizacionais de um programa são estruturadas num conjunto de etapas separadas por transições. A cada etapa está associado um conjunto de ações e a cada transição está associada uma receptividade, a qual terá que ser satisfeita para a transição ocorrer e o sistema evoluir para a etapa seguinte (Roque, 2014). Ao ligar o CLP pela primeira vez, sempre há uma etapa inicial que estará ativada, permanecendo nesse estado até que uma receptividade seja satisfeita, permitindo a evolução do sistema.

Na Figura 6 encontra-se uma representação de um sistema com as principais partes componentes de um GRAFCET.

Figura 6 - Elementos básicos do GRAFCET



Fonte: Franchi *et al*. (2020)

O GRAFCET é um gráfico fechado e cíclico (grafo) composto de etapas, transições e arcos orientados. Os arcos ligam etapas a transições e vice-versa. Uma etapa pode ter determinado número de ações associadas e as transições sempre têm

uma receptividade associada. Uma receptividade é dada na forma de uma expressão lógica e está sempre relacionada a certa transição (Franchi *et al.* , 2020).

Um dos softwares mais usados na programação do GRAFCET é o Codesys que é baseado na norma IEC 61131-3.

2.6 Conversão GRAFCET/Ladder

Segundo Franchi *et al.* (2020), antes de começar a modelar um sistema, é fundamental ter compreendido exatamente como o processo funciona. O Ladder basicamente, permite sintetizar de forma lógica, rápida e eficiente um sistema sequencial modelado em GRAFCET, com isso em mente, é possível seguir um sequenciamento de procedimentos em um projeto a ser desenvolvido, como descrito abaixo:

a) GRAFCET descritivo: o foco está na descrição lógica do sistema, sem preocupação com detalhes de implementação. As etapas de operação são escritas em sequência, cada uma com uma identificação única. O objetivo é representar o funcionamento do sistema de maneira clara e textual, facilitando a compreensão do fluxo de operações.
b) Criação das tabelas de associações: são associados bits de memória auxiliar interna a cada transição e a cada etapa. Essas associações servem como referência para a implementação do programa no CLP.
c) GRAFCET de implementação: são adicionados os detalhes de implementação, como o nome e o endereço das saídas no CLP. Essa etapa é crucial para a conversão do GRAFCET em um programa executável.
d) O programa em Ladder é desenvolvido a partir das equações das transições, etapas e ações.

A estrutura do programa deve seguir uma ordem específica para garantir o funcionamento correto do sistema:

a) ativação da etapa inicial: utiliza o bit de início de varredura (first scan), executado uma única vez;
b) detecção de bordas: identifica mudanças nas entradas (de desligadas para ligadas) e nas etapas com ações impulsionais (tipo pulso);
c) equações das transições: calcula as transições com base no estado atual e nas receptividades;

d) desativação/ativação das etapas: gerencia as etapas anteriores e posteriores às transições disparadas;
e) ativação das ações: executa as ações associadas às etapas ativas.

2.7 Codesys e a linguagem Python

A linguagem Python é uma linguagem do tipo interpretada. Em outras palavras, para a execução de um programa escrito na linguagem Python não é necessário compilar os programas, converter os programas para linguagem de máquina, desta forma, o software interpretador do Python lê os arquivos escritos pelo programador ou em uma estrutura intermediária e os executa comando a comando conforme sua estrutura lógica (Neto *et al.* , 2022).

Ela também implementa os conceitos mais modernos das linguagens de programação, por exemplo: programação orientada a objetos com herança múltipla, tratamento de erros e exceções e gerenciamento automático da memória. A isso, junta-se uma comunidade extremamente ativa, que disponibiliza um vasto conjunto de bibliotecas para, por exemplo, manipulação e tráfego de dados. As bibliotecas são pacotes de códigos prontos.

Já o Codesys é uma plataforma de software projetada para aplicações de automação industrial. O Codesys segue o padrão IEC 61131-3, garantindo compatibilidade e interoperabilidade com diversos sistemas de controle industrial. Essa plataforma oferece um conjunto completo de soluções integradas para a engenharia de projetos e o desenvolvimento de funcionalidades de automação (Filimonov *et al*. , 2024, tradução nossa).

No Codesys, é possível programar um diagrama SFC e exportá-lo como um arquivo no formato XML (*eXtensible Markup Language*, em português Linguagem de Marcação Extensível). O XML é um formato de texto estruturado que simplifica a organização e representação de dados, permitindo a padronização e o transporte de informações entre diferentes sistemas e plataformas.

2.8 Interface Web

Uma interface Web é a visualização que o usuário tem de um sistema/software que é conectado com um servidor com a finalidade de interação do usuário com a ferramenta.

Assim como são necessárias linguagens de programação para o desenvolvimento de programas e aplicativos, também é necessária uma linguagem para criação de páginas de sites. Essa linguagem é denominada HTML (*HyperText Markup Language*, em português Linguagem de Marcação de Hipertexto). Porém, em vez de comandos para execução de cálculos ou processamento de dados, ela possui marcadores, conhecidos como tags, que são utilizados para formatar um texto, uma imagem ou qualquer outro objeto que faça parte da estrutura de um documento HTML. (Alves, 2021)

Já para estilizar o conteúdo de uma interface/página Web é preciso utilizar o CSS (*Cascading Style Sheets*, em português Folhas de Estilo em Cascatas).
Pode-se considerar que a linguagem de folhas de estilo CSS diz como o conteúdo de um documento HTML deve ser exibido ao usuário, e não o que deve ser mostrado. Ela não possui comandos de execução de operações, apenas regras de formatação para serem associadas aos diversos elementos que formam uma página HTML. Essas regras têm sua própria sintaxe. (Alves, 2021)

A parte da lógica responsável por implementar uma ação, fazer uma requisição, executar instruções ou até enviar dados para uma nuvem, por exemplo, é feita utilizando linguagens de programação, como JavaScript, Python e PHP.

3 METODOLOGIA

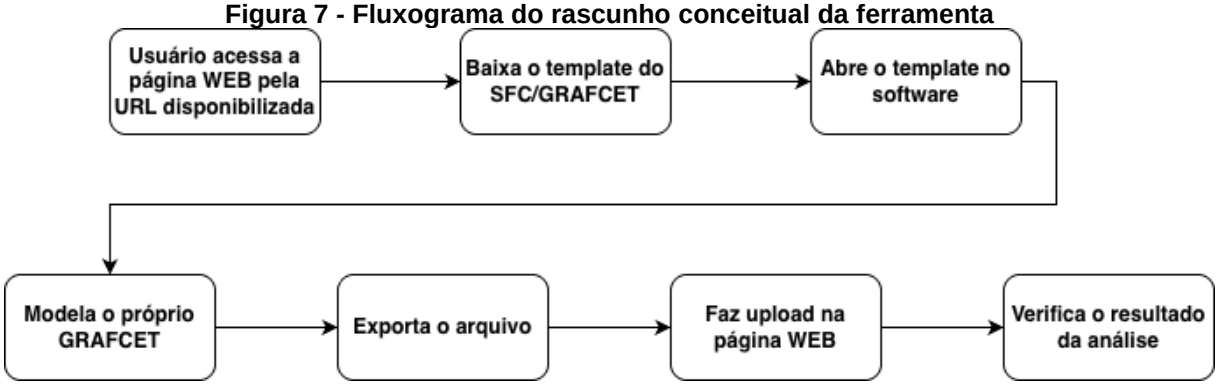
Os métodos de pesquisa relacionados aos temas do trabalho e os passos a serem seguidos durante os estudos para a obtenção da ferramenta são descritos e ilustrados.

3.1 Etapas do desenvolvimento da ferramenta

O desenvolvimento da ferramenta orientativa foi dividida em alguns passos e está diretamente relacionada aos resultados da pesquisa, abrangendo a análise do conteúdo da disciplina de manufatura integrada e decisões essenciais, como a escolha do software para criar o template do SFC/GRAFCET, a definição do método de transmissão de dados e a seleção da linguagem de programação para o programa responsável por analisar o GRAFCET e gerar orientações.

Antes do início da implementação, foi elaborado um rascunho conceitual do funcionamento da ferramenta, de acordo com o fluxograma da Figura 7, que ilustra a sequência lógica de ações a serem realizadas pelo usuário e pelo sistema.

Figura 7 - Fluxograma do rascunho conceitual da ferramenta



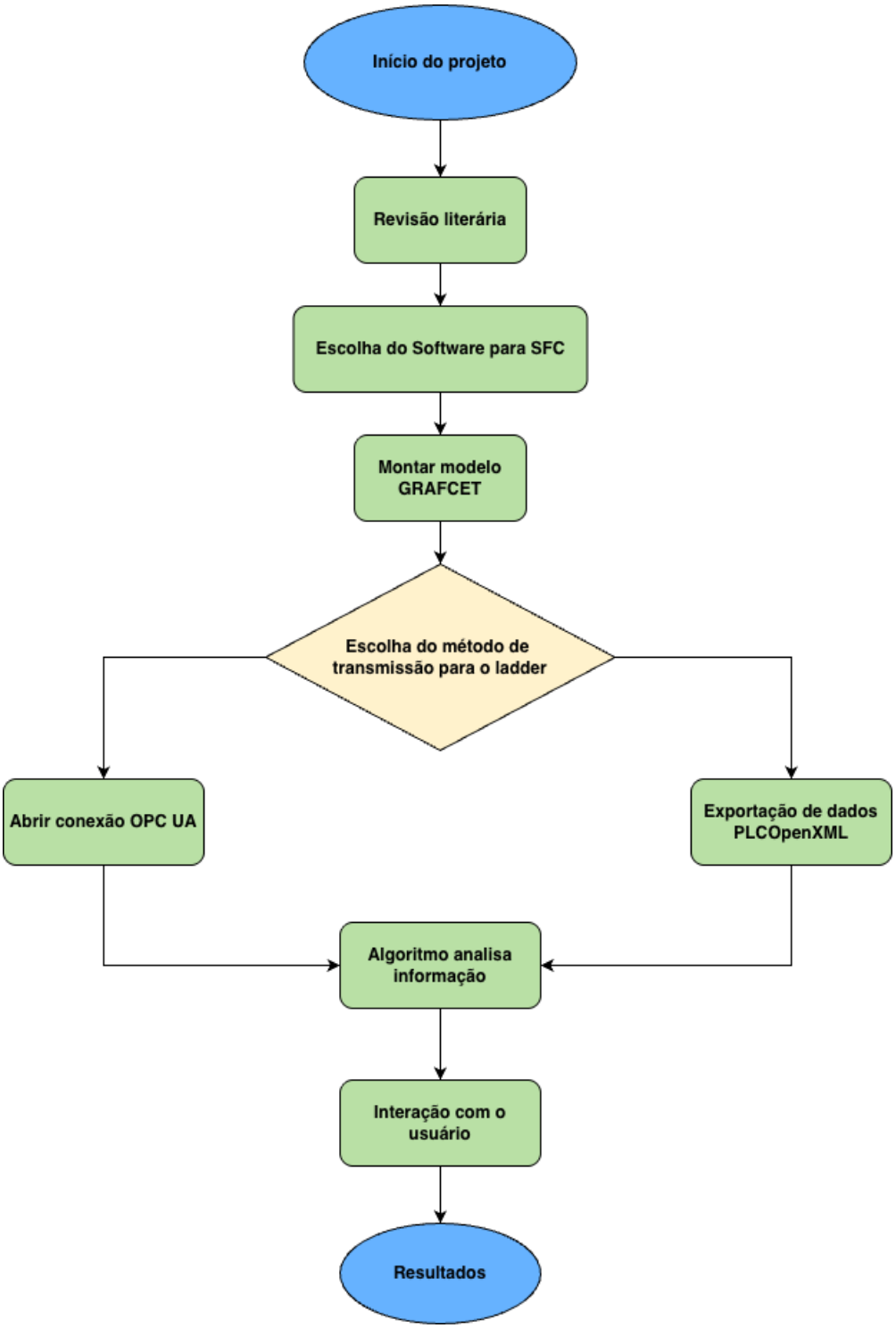
Fonte: Autoria própria (2025)

Como a proposta é ser uma ferramenta orientativa, uma página web é disponibilizada para o estudante, contendo orientações para fazer um bom GRAFCET. Nessa página, é possível baixar um template previamente criado que serve como base para o modelo desenvolvido pelo aluno, permitindo posteriormente o upload que será analisado automaticamente pela ferramenta.

Com o objetivo de organizar o processo de concepção foi elaborado um fluxograma adicional com os passos necessários para que a ferramenta tome forma,

a Figura 8 apresenta o fluxograma do processo de pesquisa e desenvolvimento do projeto.

Figura 8 - Fluxograma para o desenvolvimento da ferramenta orientativa



Fonte: Autoria própria (2025)

3.2 Revisão literária

A revisão literária tem como base o projeto final da disciplina de Manufatura Integrada ministrada pelo professor e orientador desta pesquisa, o Dr. Sidney Carlos Gasoto, mostrada no Anexo A.

O projeto final da disciplina tem como objetivo integrar de maneira eficiente os diferentes sistemas que compõem o FMS, incluindo controle, transporte, supervisão, processamento, inspeção e armazenamento.

O ciclo do processo inicia-se com a retirada da matéria-prima de uma posição do armazém, seguida do transporte até a esteira. A peça é então movida pela esteira até o torno, onde ocorre a simulação do processo de carga e descarga, incluindo a abertura e fechamento das portas do torno e a fixação da peça na castanha. Em seguida, a peça é levada ao sistema de supervisão por visão para a inspeção dimensional. Após a análise, o sistema supervisório recebe a informação sobre a aprovação ou reprovação da peça. Por fim, a peça é transportada até a frente do AS/RS e armazenada em uma das posições disponíveis, concluindo o ciclo de produção.

O conteúdo base da disciplina abrange noções básicas sobre programação Ladder, algoritmos e programação estruturada em SFC, uso das ferramentas de programação, CLP, conceitos introdutórios de visão de máquina e o emprego de ferramentas específicas dessa tecnologia.

No laboratório de FMS, o CLP utilizado é o da marca OMRON. Para programá-lo, é importante compreender seu funcionamento, incluindo entradas, saídas, e outras funções, assim como o uso do software CX Programmer para a programação em Ladder.

3.3 Software para modelagem do SFC

A escolha do software desempenha um papel fundamental no desenvolvimento da ferramenta, sendo o ponto de partida para todo o processo.

Ao levar em conta as limitações do laboratório relacionadas à memória do CLP e do Cx Programmer, os softwares considerados e avaliados foram Grafcet Studio da MHJ software e Codesys da GmbH.

O Grafcet Studio é padronizado pela DIN EN 60848 e é um software de fácil aprendizado. Oferece funcionalidades práticas, como copiar e colar, além de preenchimento automático. Também possibilita a integração com simuladores, permitindo testar e visualizar facilmente como a lógica do GRAFCET se comporta. É possível a criação de templates no software.

O Codesys, por sua vez, segue a norma IEC 61131-3 e é distribuído como software de código aberto, com licenças gratuitas e sem custos de renovação, tornando-o uma alternativa mais econômica e acessível à comunidade acadêmica. Assim como o Grafcet Studio, permite a criação de templates próprios, contando com documentação detalhada para orientar esse processo. Adicionalmente, sua instalação pode ser realizada diretamente na máquina do estudante, favorecendo o desenvolvimento remoto sem a necessidade de acesso ao laboratório.

O fator decisivo para a escolha do Codesys foi o suporte ao formato PLCopenXML, indispensável ao processamento automático do modelo dentro da ferramenta proposta. O Grafcet Studio não oferece compatibilidade com esse formato, inviabilizando sua integração ao algoritmo desenvolvido.

3.4 Desenvolvimento do modelo GRAFCET

A partir da escolha do Codesys como plataforma de desenvolvimento, foi elaborado um template específico para apoiar os estudantes na modelagem GRAFCET. Esse template funciona como ponto de partida estruturado para a criação dos modelos, fornecendo uma base organizada sobre a qual o aluno pode desenvolver seu projeto.

O template foi configurado de modo a orientar a definição correta de etapas, transições, ações, temporizadores (*timers*) e demais elementos essenciais à lógica sequencial. O objetivo não é oferecer uma solução pronta, mas sim estimular a autonomia, o raciocínio lógico e a tomada de decisões, favorecendo a aprendizagem ativa e a aplicação prática dos conceitos trabalhados na disciplina.

Além disso, o template contém instruções introdutórias e comentários explicativos incorporados diretamente ao ambiente do Codesys, auxiliando o estudante durante o processo de construção do modelo. A Figura 9 apresenta o comentário inicial incluído no template, com diretrizes para sua utilização no desenvolvimento do projeto.

Figura 9 - Visualização do POU principal PLC_PRG do template no Codesys

```
PLC_PRG.A_Etapa1 | PLC_PRG | PLC_PRG.A_Init | PLC_PRG.A_Etapa3 | PLC_PRG.A_Etapa2

3   | TEMPLATE ORIENTATIVO PARA A CONSTRUÇÃO DO GRAFCET/SFC DO PROJETO FINAL  |
4   |==========================================================================|
5   | Instruções:                                                              |
6   | 1. Cada etapa representa um estado do seu processo no FMS;               |
7   | 2. Adicione novas etapas e transições conforme o seu projeto;            |
8   | 3. Nomeie as etapas e variáveis de forma descritiva;                     |
9   | 4. Se preferir comente o que cada etapa faz (isso ajuda muito!!)         |
10  | 5. Ao finalizar:                                                         |
11  |    - Vá em "Projeto" -> "Exportar" -> "Exporta PLCOpenXML"               |
12  |    - Escolha apenas o POU principal (PLC_PRG)                            |
13  |    - Salve como <nome_do_projeto>.xml                                    |
14  | 6. Faça o upload no site para validação                                  |
15  |==========================================================================|
16  *)
17
18  PROGRAM PLC_PRG
19
20  VAR
21      // Comandos básicos
22
23      Start_Processo       : BOOL := FALSE; // Inicia o processo
```

Fonte: Autoria própria (2025)

3.5 Escolha do método de transmissão/exportação

Após a finalização da modelagem do GRAFCET no Codesys, torna-se necessário transmitir os dados estruturados do modelo para o algoritmo desenvolvido em Python, permitindo a verificação automática de consistência e identificação de possíveis erros em etapas, transições e ações. Para essa transmissão, foram considerados dois métodos: a exportação de dados no formato PLCopenXML ou o estabelecimento de comunicação via protocolo OPC UA (*Open Platform Communications Unified Architecture*).

O PLCOpenXML permite exportar um arquivo baseado em XML que descreve objetos no formato PLCOpen e permite a troca de projetos completos ou partes de projetos (POUs - *programming organization unit,* em português Unidade de Organização de Programa) entre softwares de programação de CLPs de diferentes fornecedores que suportam o padrão IEC 61131-10.

Para o algoritmo em Python ler esse arquivo é preciso utilizar de métodos e funções da própria linguagem de programação ou de módulos externos, como a ElementTree, que é uma biblioteca Python que analisa dados XML.

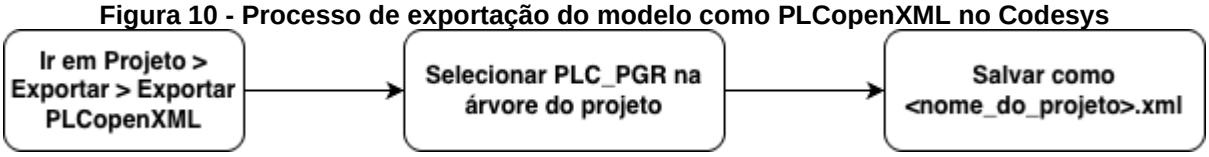
Outra forma de interação é estabelecer uma comunicação entre Python e Codesys por meio do protocolo OPC UA. A comunicação ocorre em tempo real,

estabelecendo uma arquitetura cliente-servidor, onde o ambiente Python atua como cliente OPC UA e o controlador (virtual ou físico) executado no Codesys atua como servidor. Essa abordagem permite transmissão contínua e escalável de dados, sendo amplamente utilizada em aplicações industriais.

Entretanto, apesar de seu potencial, a implementação do OPC UA exigiria a configuração de módulos adicionais (*add-ons*) dentro do Codesys, além de etapas complexas de parametrização de segurança, certificados e estrutura de endereçamento de variáveis. Esses fatores tornam o processo mais demorado e inadequado ao contexto didático e ao objetivo deste trabalho, que busca acessibilidade e simplicidade para favorecer o aprendizado dos estudantes.

Dessa forma, optou-se pela exportação em PLCopenXML, por se tratar do método mais simples, direto e padronizado, além de oferecer total compatibilidade com o algoritmo desenvolvido. A Figura 10 apresenta o fluxograma que representa o processo de exportação do arquivo XML.

Figura 10 - Processo de exportação do modelo como PLCopenXML no Codesys



Fonte: Autoria própria (2025)

3.6 Desenvolvimento do algoritmo de processamento e análise

O algoritmo para processamento das informações e análise dos dados foi codificado com a linguagem Python no ambiente de desenvolvimento *Visual Studio Code*. O projeto, denominado grafcet_analyser, foi estruturado de forma modular, integrando dois componentes principais: o script de análise e a página web destinada à interação com o usuário.

O objetivo central do algoritmo é verificar a consistência estrutural do modelo desenvolvido pelo estudante, assegurando que o projeto atenda aos requisitos mínimos de funcionamento lógico esperados na disciplina. Para isso, o algoritmo realiza verificações automáticas como a existência e correta associação de etapas (*steps*), ações, transições e conexões válidas, identificando erros ou omissões que possam comprometer o comportamento sequencial do sistema.

Essa abordagem permite que o estudante receba um diagnóstico preliminar do modelo antes da implementação no CLP. Além disso, a integração entre o

algoritmo e a interface web oferece um processo ágil de validação, permitindo que o usuário carregue o arquivo XML e visualize imediatamente mensagens de alerta e inconsistências detectadas.

3.6.1 Leitura e estruturação do arquivo XML

O XML organiza informações por meio de uma estrutura hierárquica em forma de árvore, composta por elementos identificados por tags, que podem conter atributos, valores e elementos filhos. Essa característica torna o formato adequado para descrever modelos estruturados, como projetos de automação industrial, permitindo que diferentes softwares interpretem os mesmos dados de forma padronizada.

O arquivo PLCopenXML gerado pelo Codesys contém os elementos que definem o modelo GRAFCET, como etapas (*steps*), transiçoes, ações e conexões. Cada um desses elementos é representado como um nó na estrutura da árvore XML, possibilitando a navegação e extração de suas propriedades pelo algoritmo de análise.

Para realizar a leitura e interpretação do arquivo XML, utilizou-se a biblioteca nativa do Python xml.etree.ElementTree, que converte o documento textual em uma árvore de objetos manipuláveis programaticamente. A função *parse* é responsável por carregar o arquivo e construir essa estrutura hierárquica, permitindo que o algoritmo percorra os nós e acesse suas informações para validação posterior. O trecho de código correspondente é apresentado no Quadro 2.

Quadro 2 - Método *parse* responsável por transformar o arquivo em uma estrutura de árvore python com nós individuais

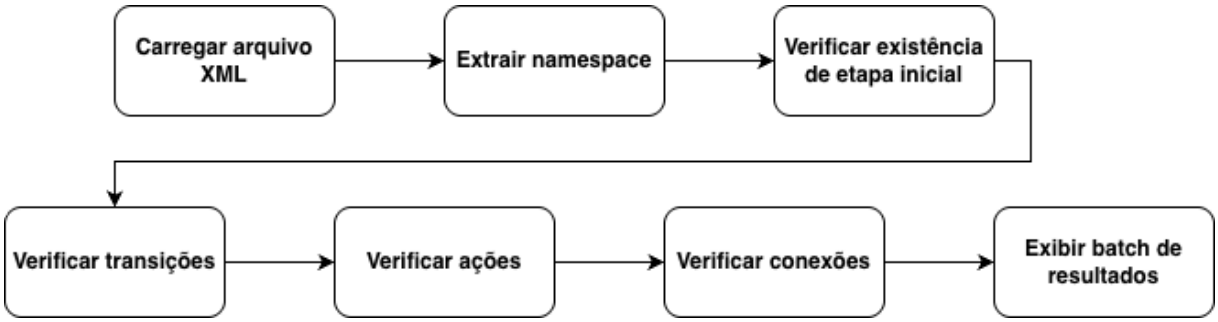
1	tree = xml.etree.ElementTree.parse(caminho_do_arquivo)

Fonte: Autoria própria (2025)

3.6.2 Estrutura e funcionamento do algoritmo

Após o carregamento do arquivo, o algoritmo executa uma série de funções de validação. Essas funções são responsáveis por extrair os elementos citados, ações, transições, steps e conexões. O processo dessas análises foi desenvolvido de acordo com o fluxograma apresentado na Figura 11.

Figura 11 - Funções executadas pelo algoritmo



Fonte: Autoria própria (2025)

O processo de validação percorre cada componente do GRAFCET, verificando, entre outros aspectos:

a) a existência de ações associadas a cada etapa;
b) a correção das transições, assegurando que possuam condições definidas;
c) a coerência das conexões, garantindo início e destino válidos;
d) a presença de uma etapa inicial e ausência de etapas isoladas.

3.7 Interação com o usuário

A interação com o usuário é o elemento central da ferramenta educacional proposta, pois garante que os alunos recebam orientações claras e construtivas durante o processo de modelagem e análise. O intuito não é oferecer respostas prontas e sim ser uma ferramenta que auxilie no aprendizado e planejamento do projeto final, evitando que o aluno pule fases.

Para viabilizar essa comunicação, foi desenvolvida uma interface web interativa, construída utilizando HTML e CSS para a estrutura e o design visual, e o framework Flask, em Python, responsável por integrar o algoritmo de análise à aplicação web. Essa arquitetura possibilita a criação de um ambiente acessível por qualquer navegador, dispensando instalações adicionais e permitindo o uso tanto em laboratório quanto em dispositivos pessoais.

Durante o processo de concepção da interface, foram definidos os seguintes requisitos funcionais:

a) deve possuir etapas orientativas para construção de um GRAFCET;
b) disponibilizar um template base no Codesys que servirá como ponto de partida para o estudante;

c) explicar o propósito e as limitações do analisador, reforçando seu caráter educativo;
d) permitir que o usuário faça upload do arquivo no formato XML gerado pelo Codesys;
e) exibir os resultados da análise de forma compreensível, com mensagens categorizadas em sucesso, alerta e erro.

Com o intuito de permitir acesso remoto e facilitar a adoção da ferramenta em ambiente educacional, a aplicação web foi hospedada na plataforma Render, um serviço de deploy em nuvem que oferece suporte nativo a aplicações. A escolha do Render se deve à sua praticidade de configuração, integração contínua com repositórios Git e possibilidade de disponibilização pública da ferramenta sem custos iniciais.

Entretanto, por se tratar de um plano gratuito, o ambiente é desativado automaticamente após um período de inatividade, exigindo que o responsável pela aplicação realize manualmente o processo de reativação e deploy sempre que houver necessidade de atualização ou quando o serviço for temporariamente suspenso.

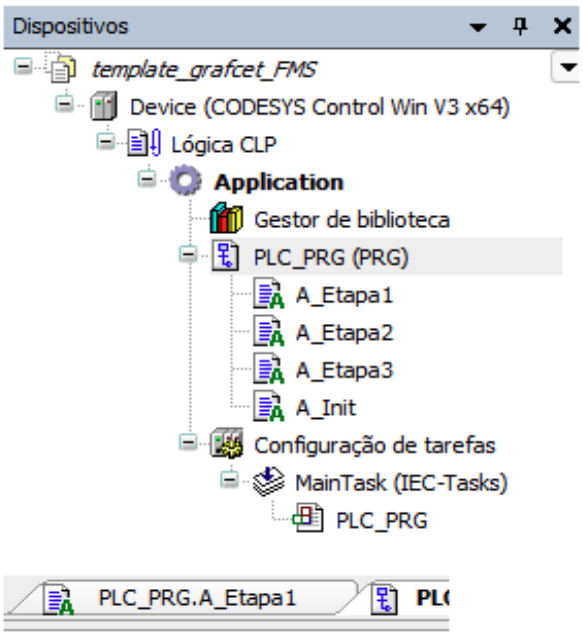
4 ANÁLISE DE RESULTADOS

Este capítulo apresenta os resultados obtidos a partir do desenvolvimento da ferramenta orientativa. São descritos os componentes criados, o código desenvolvido, os testes realizados, assim como as respostas retornadas pela aplicação, com o intuito de demonstrar o desempenho e a utilização do sistema no contexto educacional.

4.1 Organização do template e estruturação do modelo GRAFCET

A estrutura do template foi planejada para auxiliar o estudante na construção do modelo de forma didática e progressiva, contemplando exemplos de referência para facilitar a compreensão sem oferecer soluções prontas. A Figura 12 apresenta a árvore de projeto no Codesys, evidenciando a organização dos elementos principais do template, incluindo o arquivo PLC_PRG, que corresponde ao diagrama SFC (POU) exportado e posteriormente analisado pela ferramenta.

Figura 12 - Árvore do projeto no Codesys



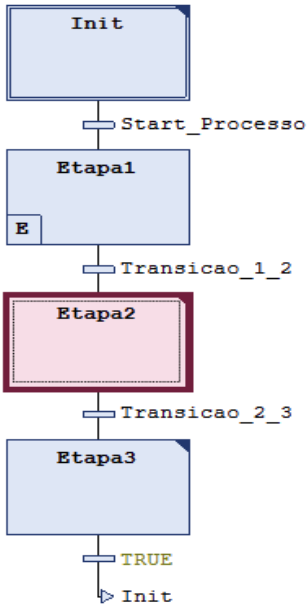
Fonte: Autoria própria (2025)

Logo abaixo do arquivo principal PRG, encontram-se os objetos de ação, no formato ST, texto estruturado, que representam as ações associadas a cada etapa do GRAFCET. É importante que o aluno siga a mesma nomenclatura para a criação de novas etapas.

O Apêndice A apresenta o código principal da aplicação, no qual foram declaradas algumas variáveis de exemplo, distribuídas entre comandos básicos, variáveis de estado, condição de transição, sensores/atuadores e *timers* com o intuito de mostrar as possibilidades de uso.

Após a declaração das variáveis, o SFC foi estruturado graficamente no PRG, utilizando blocos representativos das etapas e transições, conforme ilustrado na Figura 13.

Figura 13 - SFC estruturado em etapas e transições



Fonte: Autoria própria (2025)

4.2 Processamento e interpretação do arquivo XML

Durante o desenvolvimento do algoritmo, identificou-se a necessidade de extrair corretamente o *namespace* presente no arquivo PLCopenXML. Essa identificação é essencial, pois o *namespace* define o contexto e a origem das tags utilizadas no projeto, garantindo que o programa consiga localizar corretamente os elementos internos que precisam ser analisados.

Foi criada uma função dedicada à extração do *namespace*, capaz de interpretar automaticamente a estrutura XML e isolar o valor do atributo xmlns da tag principal <project>. Um exemplo dessa estrutura é apresentado no Quadro 3, onde é possível visualizar o *namespace* especificado no início do arquivo.

Quadro 3 - Exemplo de namespace na tag principal <project>

1	<?xml version="1.0" encoding="utf-8"?>
2	<project xmlns="http://www.plcopen.org/xml/tc6_0200">
3	… // resto do código
4	</project>

Fonte: Autoria própria (2025)

Essa implementação solucionou limitações iniciais relacionadas à incompatibilidade de endereçamento das tags e permitiu o correto funcionamento do módulo de análise. Assim, o resultado atende ao objetivo de permitir a leitura estruturada do XML para análises automatizadas, viabilizando o processamento completo do modelo por meio do algoritmo.

Após a extração do namespace, o algoritmo inicia o processo de verificação das estruturas internas do arquivo.

4.3 Validação automática do modelo

A primeira função executada é a de verificação da etapa inicial, que é responsável por identificar se o módulo possui exatamente uma etapa inicial. Nessa etapa, o algoritmo busca todas as tags <step> que contenham o atributo de step inicial.

Em seguida, é verificado se existem transições vazias, ou seja, o algoritmo percorre todas as tags <transition> do arquivo e analisa se possuem uma condição de transição definida, seja por texto direto, bloco lógico ou referência a outro elemento. Essa transição vazia não é necessariamente um erro, mas sim um alerta, para que o aluno verifique o modelo e corrija, uma vez que transições sem condições explícitas podem comprometer o comportamento lógico.

Para verificar as ações, foi criada uma função com o objetivo de confirmar se todas as etapas possuem ações. Ela atua diretamente sobre os elementos <step> e busca referências de ação associadas a cada etapa. Também é possível fazer a análise inversa, identificar as ações que foram declaradas mas não utilizadas por nenhuma etapa.

Para validar se todas as transições possuem conexões válidas de origem e destino, o algoritmo coleta todos os identificadores (localId) presentes no XML e verifica se cada transição está corretamente conectada.

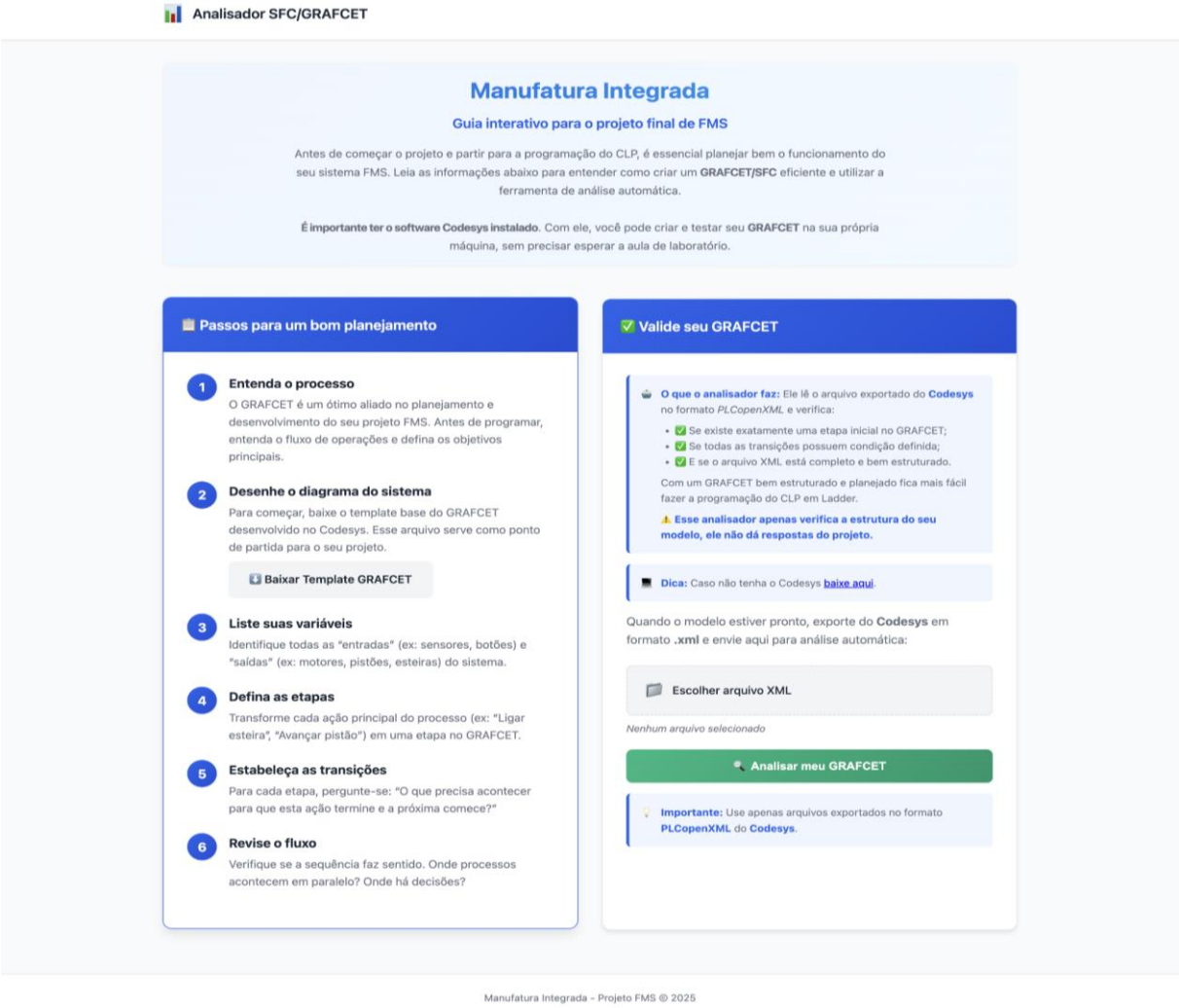
Por fim, para verificar as conexões foi criada uma função responsável por validar as ligações internas. Todas as conexões devem fazer referência a elementos existentes. A comparação é feita conferindo se o atributo refLocalId de cada <connection> está presente no conjunto de identificadores válidos do projeto.

De acordo com cada verificação é emitido um texto alertando o usuário do status da análise. É possível visualizar o código completo do analisador no Apêndice B.

4.4 Teste da ferramenta com modelos válidos e inválidos

A Figura 14 apresenta a página inicial da aplicação web, na qual o usuário encontra instruções sobre o uso da ferramenta e as opções para baixar o template e enviar o arquivo XML.

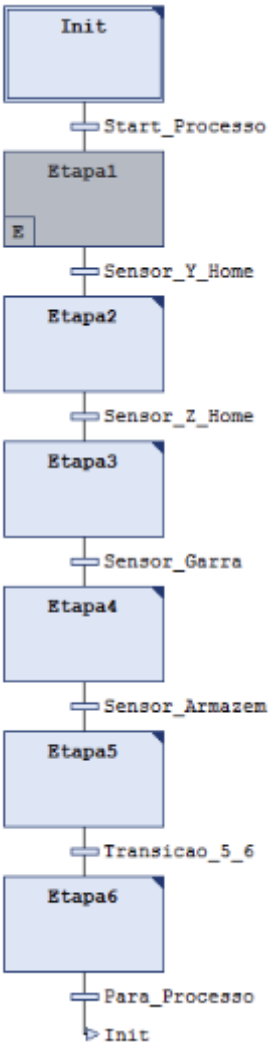
Figura 14 - Página web da ferramenta orientativa



Fonte: Autoria própria (2025)

Para testar a ferramenta criou-se um GRAFCET de testes com base no template disponibilizado no site. Foram adicionadas mais algumas etapas, assim como novas variáveis. Optou-se por seguir com a mesma nomenclatura aplicada no template em relação às variáveis e ações. A visualização gráfica do SFC é mostrada na Figura 15.

Figura 15 - SFC criado com base no template disponibilizado



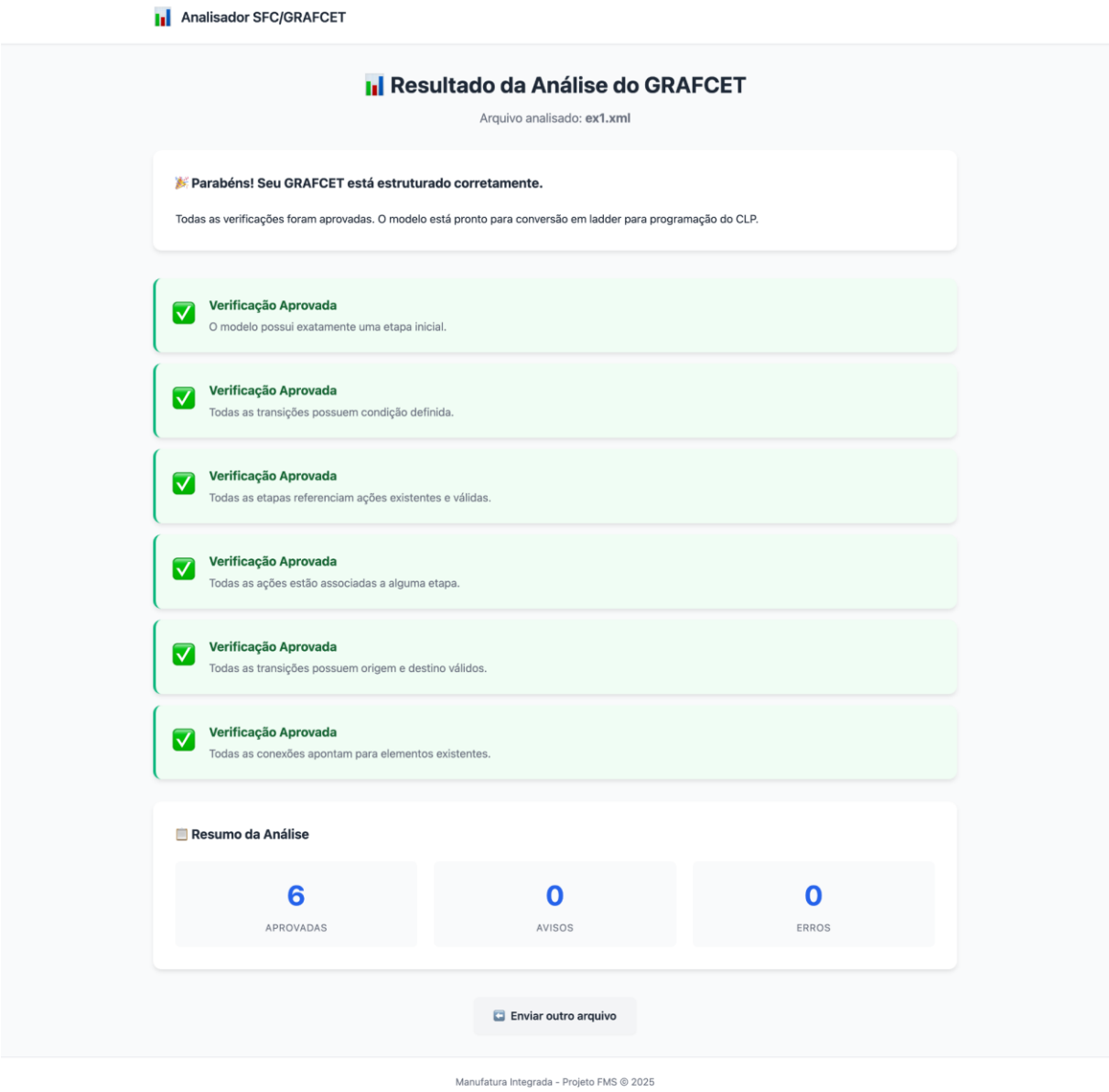
Fonte: Autoria própria (2025)

Ao finalizar o modelo, o arquivo foi exportado e carregado no site.

A visualização dos resultados da análise é apresentada na Figura 16, que exibe a interface responsável por retornar ao usuário o diagnóstico do arquivo GRAFCET submetido. Nessa página, são mostradas as mensagens geradas pelo algoritmo, classificadas conforme o tipo de ocorrência, permitindo ao aluno

compreender de forma clara quais aspectos do modelo estão corretos e quais necessitam de ajustes.

Figura 16 - Página de visualização dos resultados



Fonte: Autoria própria (2025)

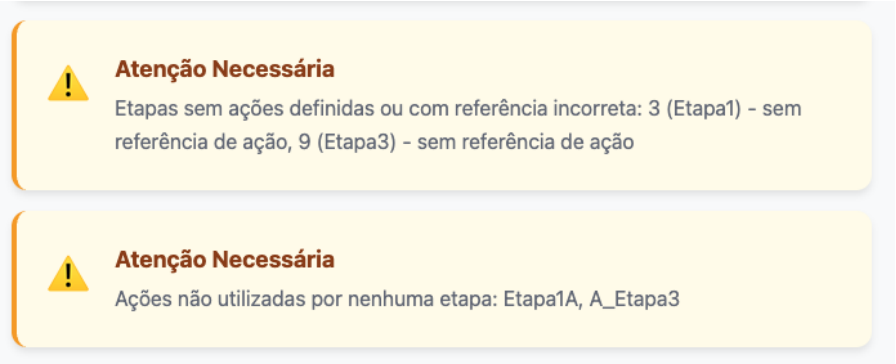
Após a validação positiva foi realizado um teste com um projeto propositalmente configurado com erros, a fim de verificar a capacidade do algoritmo em detectar inconsistências. No Codesys, algumas ações foram modificadas e outras excluídas, resultando em alertas gerados pelo analisador. Isso ocorre porque o algoritmo identifica as ações com base em uma convenção de nomenclatura, na qual

o nome da ação é concatenado ao número da etapa correspondente. Assim, qualquer desvio desse padrão é automaticamente sinalizado nos resultados.

Além disso, o sistema também foi capaz de detectar etapas sem ações associadas, demonstrando a eficácia das funções de verificação implementadas.

Esses alertas podem ser observados na Figura 17, que apresenta os alertas emitidos após a validação.

Figura 17 - Componente de visualização dos resultados - alerta



Fonte: Autoria própria (2025)

5 CONCLUSÃO

Este trabalho teve como objetivo geral desenvolver uma ferramenta educacional orientativa para apoiar estudantes no processo de elaboração e análise de modelos GRAFCET, contribuindo para a aprendizagem prática da disciplina de Manufatura Integrada. A partir da análise dos resultados apresentados no capítulo anterior, verifica-se que os objetivos estabelecidos foram atendidos de maneira satisfatória.

Os resultados obtidos demonstram que o algoritmo de análise, em conjunto com o template base, foi capaz de identificar a maioria das inconsistências estruturais nos arquivos XML gerados pelo Codesys, como ausência de etapas iniciais, ações não referenciadas ou inexistentes e conexões inválidas.

Durante os testes, entretanto, foram observadas limitações no processo de verificação, especialmente em relação às transições. Em um modelo de teste propositalmente modificado com erros, a remoção de determinadas transições não resultou em alertas durante a análise. Esse comportamento evidencia a necessidade de aperfeiçoar os critérios de validação, tornando o algoritmo mais robusto na detecção de exclusões e modificações estruturais.

Além disso, a dependência exclusiva da análise de tags XML impõe restrições ao projeto, sobretudo em cenários mais complexos, com múltiplas transições, ações condicionais e interações avançadas. Como todo o processo é baseado na leitura e interpretação dessas tags, o escopo atual da ferramenta limita-se à estrutura textual do modelo, não alcançando representações visuais ou funcionais mais complexas.

A interface web disponibilizada possibilitou uma interação simples e acessível, contribuindo para a adoção prática da solução tanto dentro quanto fora do laboratório. O ambiente de uso remoto tornou viável o teste e a evolução dos modelos sem dependência exclusiva da estrutura física da instituição, ampliando o alcance e a aplicabilidade da proposta.

Entre as melhorias futuras, destaca-se a possibilidade de integrar a ferramenta a outros ambientes de desenvolvimento, como o CX-Programmer, utilizado para programação em Ladder de controladores Omron. Essa integração permitiria importar modelos GRAFCET e convertê-los automaticamente em Ladder, ampliando significativamente o alcance e a aplicabilidade da solução, e reduzindo mais um passo no desenvolvimento do projeto final da disciplina.

Também se sugere a ampliação dos templates disponíveis na página web, adaptando-os a diferentes tipos de projetos propostos pelo professor. Quanto à hospedagem, atualmente feita na plataforma Render em ambiente gratuito, há a limitação de reativação manual a cada novo acesso. Migrar para uma plataforma de hospedagem estável pode oferecer maior disponibilidade, além de facilitar a atualização de templates e o gerenciamento do conteúdo diretamente pela interface, sem a necessidade de reedição no Visual Studio Code.

REFERÊNCIAS

FILIMONOV, D.; ONABEK, A.; SMOLYARCHUK, K.; ALIZADEH, T. Integrating Computer Vision in a CODESYS PLC to Enable Intelligent Object Identification**.** In: 2024 9th INTERNATIONAL CONFERENCE ON MECHATRONICS ENGINEERING (ICOM). Kuala Lumpur, Malaysia. **Proceedings [...].** p. 65-70, doi: 10.1109/ICOM61675.2024.10652332. Disponível em: https://ieeexplore.ieee.org/abstract/document/10652332. Acesso em: 20 jan. 2025.

FRANCHI, M.; CAMARGO, V. **Controladores Lógicos Programáveis.** 3. ed. Rio de Janeiro: Érica, 2020. E-book. p.37. ISBN 9788536533605. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9788536533605/. Acesso em: 20 jan. 2025.

GROOVER, P. **Fundamentos da Moderna Manufatura** - Vol. 2, 5ª edição. Rio de Janeiro: LTC, 2017. E-book. p.i. ISBN 9788521634102. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9788521634102/. Acesso em: 10 dez. 2024.

MORAES, M. **Robótica Educacional:** socializando e produzindo conhecimentos matemáticos. 2010. 144 f. Dissertação (Mestrado em Educação em Ciências) – Universidade Federal do Rio Grande, Rio Grande do Sul. 2010.

NETO, T.; SILVA, F. **Introdução à Programação para Engenharia:** Usando a Linguagem Python. Rio de Janeiro: LTC, 2022. E-book. p.29. ISBN 9788521638346. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9788521638346/. Acesso em: 20 jan. 2025.

PIAGET, J. **A construção do real na criança**. Rio de Janeiro: Zahar, 1970.

ROQUE, L. **Automação de Processos com Linguagem Ladder e Sistemas Supervisórios**. Rio de Janeiro: LTC, 2014. E-book. ISBN 978-85-216-2683-1. Disponível em: https://app.minhabiblioteca.com.br/reader/books/978-85-216-2683-1/. Acesso em: 10 dez. 2024.

ALVES, P. **HTML & CSS:** aprenda como construir páginas web. Rio de Janeiro: Expressa, 2021. E-book. p.7-28. ISBN 9786558110187. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9786558110187/. Acesso em: 02 out. 2025.

APÊNDICE A - Código do PLC_PRG do template criado no codesys

```
1   PROGRAM PLC_PRG
2
3   VAR
4       // Comandos básicos
5       Start_Processo      : BOOL := FALSE; // Inicia o processo
6       Para_Processo       : BOOL := FALSE; // Para o processo
7
8       // Variáveis de estado
9       Etapa1_Ativa        : BOOL := FALSE;
10      Etapa2_Ativa        : BOOL := FALSE;
11      Etapa3_Ativa        : BOOL := FALSE;
12
13      // Condição de transição
14      Transicao_1_2       : BOOL := FALSE;
15      Transicao_2_3       : BOOL := FALSE;
16      Transicao_3_Init    : BOOL := FALSE;
17
18      // Sensores e/ou atuadores
19      Sensor_Y_Home       : BOOL := FALSE;
20      Sensor_Z_Home       : BOOL := FALSE;
21      Sensor_Garra        : BOOL := FALSE;
22      Motor1              : BOOL := FALSE;
23      Motor2              : BOOL := FALSE;
24      Valvula1            : BOOL := FALSE;
25
26      // Timers simulados
27      Timer_11            : TON;
28      Timer_25            : TON;
29
30  END_VAR
31  // Após criar suas variáveis você pode dar dois cliques na
```

32	// etapa abaixo e criar a lógica (Ação) correspondente para essa etapa
33	
34	// Na Caixa de Ferramentas ao lado direito você consegue adicionar mais etapas,
35	// transições, ramificações, etc.

APÊNDICE B - Código do algoritmo analisador em python

```
import xml.etree.ElementTree as ET

def extrair_namespace(root):
    if '}' in root.tag:
        return root.tag.split('}')[0][1:]
    return ‘ ‘

def verificar_etapa_inicial(root, ns):
    path = f".//{{{ns}}}step[@initialStep='true']"
    initial_steps = root.findall(path)
    if len(initial_steps) == 0:
        return {'status': 'error', 'message': 'O modelo não possui nenhuma etapa inicial.'}
    if len(initial_steps) > 1:
        names = [s.get('name', 'SemNome') for s in initial_steps]
        return {'status': 'error', 'message': f'Há {len(initial_steps)} etapas iniciais: {", ".join(names)}.'}
    return {'status': 'success', 'message': 'O modelo possui exatamente uma etapa inicial.'}

def verificar_transicoes_vazias(root, ns):
    path = f".//{{{ns}}}transition"
    transicoes = root.findall(path)
    sem_condicao = []
    for trans in transicoes:
        cond = trans.find(f"./{{{ns}}}condition")
        if cond is None:
            sem_condicao.append(trans.get("localId", "SemID"))
            continue

        inline_body = cond.find(f".//{{{ns}}}body")
        if inline_body is not None and inline_body.text and inline_body.text.strip():
            continue
```

```
31
32        connection = cond.find(f".//{{{ns}}}connection")
33        if connection is not None and connection.get("refLocalId"):
34            continue
35
36        sem_condicao.append(trans.get("localId", "SemID"))
37
38        if sem_condicao:
39            return {'status': 'warning', 'message': f'Transições sem condição explícita: {",
".join(sem_condicao)}'}
40        return {'status': 'success', 'message': 'Todas as transições possuem condição definida.'}
41
42    def verificar_etapas_sem_acao(root):
43        acoes_definidas = [a.get("name") for a in root.iter() if a.tag.endswith("action")]
44
45        etapas = [e for e in root.iter() if e.tag.endswith("step")]
46        etapas_sem_acao = []
47
48        for etapa in etapas:
49            step_id = etapa.get("localId", "?")
50            step_name = etapa.get("name", "SemNome")
51
52            atribuicoes = [attr.text for attr in etapa.iter() if attr.tag.endswith("attribute")]
53            acao_ref = None
54            for attr_text in atribuicoes:
55                if attr_text and attr_text.startswith("A_"):
56                    acao_ref = attr_text.strip()
57
58            if not acao_ref:
59                etapas_sem_acao.append(f'{step_id} ({step_name}) - sem referência de ação')
60                continue
```

```
        if acao_ref not in acoes_definidas:
            etapas_sem_acao.append(f'{step_id} ({step_name}) → ação "{acao_ref}" não encontrada')
            continue

    if etapas_sem_acao:
        return {
            'status': 'warning',
            'message': f'Etapas sem ações definidas ou com referência incorreta: {", ".join(etapas_sem_acao)}'
        }

    return {
        'status': 'success',
        'message': 'Todas as etapas referenciam ações existentes e válidas.'
    }

def verificar_acoes_orfas(root):
    acoes_definidas = [a.get("name") for a in root.iter() if a.tag.endswith("action")]
    atributos = [attr.text for attr in root.iter() if attr.tag.endswith("attribute") and attr.text and attr.text.startswith("A_")]
    acoes_usadas = set(atributos)
    acoes_orfas = [a for a in acoes_definidas if a not in acoes_usadas]

    if acoes_orfas:
        return {
            'status': 'warning',
            'message': f'Ações não utilizadas por nenhuma etapa: {", ".join(acoes_orfas)}'
        }
    return {
        'status': 'success',
        'message': 'Todas as ações estão associadas a alguma etapa.'
```

```
    }

def verificar_transicoes_orfas(root, ns):
    transicoes = root.findall(f".//{{{ns}}}transition")
    todas_ids = {el.get("localId") for el in root.findall(f".//*[@localId]")}
    conexoes = root.findall(f".//{{{ns}}}connection")

    origem_por_trans = {}
    destino_por_trans = {}

    for t in transicoes:
        local_id = t.get("localId")
        conexoes_internas = t.findall(f".//{{{ns}}}connection[@refLocalId]")
        if conexoes_internas:
            origem_por_trans[local_id] = [c.get("refLocalId") for c in conexoes_internas]

    for conn in conexoes:
        ref = conn.get("refLocalId")
        if ref in todas_ids:
            destino_por_trans.setdefault(ref, []).append(conn)

    sem_origem = []
    sem_destino = []
    for t in transicoes:
        tid = t.get("localId")
        if tid not in origem_por_trans:
            sem_origem.append(tid)
        if tid not in destino_por_trans:
            sem_destino.append(tid)

    msgs = []
```

```
    if sem_origem:
        msgs.append(f'Sem origem: {", ".join(sem_origem)}')
    if sem_destino:
        msgs.append(f'Sem destino: {", ".join(sem_destino)}')

    if msgs:
        return {
            'status': 'warning',
            'message': 'Transições possivelmente incompletas → ' + " | ".join(msgs)
        }
    return {'status': 'success', 'message': 'Todas as transições possuem origem e destino válidos.'}

def verificar_conexoes_orfas(root, ns):
    todas_ids = {el.get("localId") for el in root.findall(f".//*[@localId]")}
    orfas = []
    for conn in root.findall(f".//{{{ns}}}connection"):
        ref = conn.get("refLocalId")
        if ref and ref not in todas_ids:
            orfas.append(ref)
    if orfas:
        return {'status': 'warning', 'message': f'Conexões apontando para elementos inexistentes: {", ".join(orfas)}'}
    return {'status': 'success', 'message': 'Todas as conexões apontam para elementos existentes.'}

def analisar_arquivo_grafcet(caminho_do_arquivo):
    try:
        tree = ET.parse(caminho_do_arquivo)
        root = tree.getroot()
        ns = extrair_namespace(root)

```

```
        resultados = [
            verificar_etapa_inicial(root, ns),
            verificar_transicoes_vazias(root, ns),
            verificar_etapas_sem_acao(root),
            verificar_acoes_orfas(root),
            verificar_transicoes_orfas(root, ns),
            verificar_conexoes_orfas(root, ns)
        ]

        return {'success': True, 'resultados': resultados}

    except ET.ParseError:
        return {'success': False, 'error': 'O arquivo não é um XML válido ou está corrompido.'}
    except Exception as e:
        return {'success': False, 'error': f'Erro inesperado ao processar: {str(e)}'}
```

ANEXO A - Trabalho de Avaliação dos componentes do FMS

Disciplinas: ME78B – Sistemas Flexíveis de Manufatura
Professor: Sidney Carlos Gasoto

Trabalho de Avaliação (TA):

O trabalho consiste da programação dos seguintes componentes do FMS:

- CLP, para controle geral do FMS;
- Supervisório, para visualização dos estados e controle;
- CNC (apenas códigos M para interação com CLP);
- Sistema de visão (para inspeção de peças);
- Armazém (movimentação da matéria prima e peça acabada).

Grupos de 3 a 5 alunos.

Objetivo:
Realizar os seguintes passos de uma produção simulada de uma peça:

1. Ao start do supervisório, retirar a matéria prima de um dos nichos a escolher, e levá-la até a esteira, fazer a peça se movimentar através da esteira, saindo da frente do AS/RS e parando em um dos processadores (torno ou fresa e sistema de visão), retornar ao AS/RS e armazená-la novamente. O supervisório deve mostrar o que está acontecendo no FMS.
2. Simular o processo de usinagem da peça com pausa de 5 segundos com a peça presa na placa do torno ou morsa da fresadora, sendo os processos de carga e descarga executados normalmente e controlados pelo sistema, mostrando o estado do processo no supervisório. A carga e descarga da peça será executada por um dos robôs Mitsubish.
3. Levar a peça até o centro de inspeção, e realizar a inspeção dimensional da peça. Informar o supervisório da dimensão medida e eventual aprovação/reprovação da peça.
4. Levar a peça até a frente do AS/RS,
5. Armazenar em um dos nichos, a escolher, finalizando a produção. Gerar no supervisório um resumo (relatório) do processo realizado nesta peça (peça X, produzida na data Y, levou Z minutos, etc.)

Avaliação:

60% - nota do grupo: o projeto deve funcionar adequadamente, e corretamente. Cada falha implica redução de 10% na nota do grupo.

40% - nota individual: cada aluno deverá responder ao professor 4 perguntas sobre os códigos dos programas do grupo. Cada pergunta vale 10% da nota individual.